

A Plan for C++26 Ranges

Document #: P2214R2
Date: 2023-08-14
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Conor Hoekstra
<conorhoekstra@gmail.com>
Tim Song
<t.canens.cpp@gmail.com>

Contents

- [1 Introduction](#)
- [2 Views](#)
 - [2.1 `cache\_last`](#)
 - [2.2 `istream<T>`](#)
 - [2.3 `scan`](#)
 - [2.4 `generate`](#)
 - [2.5 `as\_input`](#)
 - [2.6 Simple Adaptor Compositions](#)
 - [2.7 Extending `conditionally\_borrowed`](#)
- [3 View Adjuncts](#)
 - [3.1 More Function Objects](#)
 - [3.2 More Function Adaptors](#)
- [4 Algorithms](#)
 - [4.1 `reduce`](#)
 - [4.2 `distance` and `advance`](#)
- [5 Output Iterators](#)
 - [5.1 Potential Design](#)
- [6 Plan Summary](#)
 - [6.1 Tier 1](#)
 - [6.2 Tier 2](#)
 - [6.3 Tier 3](#)
- [7 References](#)

§ 1 Introduction

For the C++23 cycle, we set out to create a plan to prioritize what additions we wanted to make for Ranges [P2214R2]. We ended up adopting all of the proposals we originally labelled as Tier 1 (with the exception of some we deliberately dropped, see later), as well as some from Tier 2. Moreover, based on

the questions we've seen in various contexts about how to solve certain problems with Ranges - a significant percentage of them can be answered with some new C++23 facility, which suggests that we prioritized the right tools.

To summarize, in C++23 we adopted the following facilities:

- General additions to ranges:
 - the ability to define first-class user-defined range adaptors ([\[P2387R3\]](#))
 - the ability to collect a range into a container, `ranges::to` ([\[P1206R7\]](#))
 - the ability to format ranges ([\[P2286R8\]](#))
- New range adaptors:
 - `views::adjacent` and `views::adjacent_transform` ([\[P2321R2\]](#))
 - `views::as_const` ([\[P2278R4\]](#))
 - `views::as_rvalue` ([\[P2446R2\]](#))
 - `views::cartesian_product` ([\[P2374R4\]](#))
 - `views::chunk` ([\[P2442R1\]](#))
 - `views::chunk_by` ([\[P2443R1\]](#))
 - `views::enumerate` ([\[P2164R9\]](#))
 - `views::join_with` ([\[P2441R2\]](#))
 - `views::repeat` ([\[P2474R2\]](#))
 - `views::slide` ([\[P2442R1\]](#))
 - `views::stride` ([\[P1899R3\]](#))
 - `views::zip` and `views::zip_transform` ([\[P2321R2\]](#))
- New (or improved) range algorithms:
 - allowing C++20 iterators to be used in C++17 algorithms ([\[P2408R5\]](#))
 - `ranges::contains` ([\[P2302R4\]](#))
 - `ranges::fold` and family ([\[P2322R6\]](#))
 - `ranges::iota` ([\[P2440R1\]](#))
 - `ranges::shift_left` and `ranges::shift_right` ([\[P2440R1\]](#))

There were also a bunch of smaller improvements that are not listed here.

But there's still plenty more work to be done - both on the range adaptor and the range algorithm front. The goal of this paper is to do for the C++26 timeframe what our previous plan did for the C++23 one: express what we think is the right prioritization of work, while describing what some of the outstanding issues are so that we can start tackling them.

§ 2 Views

As before, we'll start by enumerating all the adaptors in range-v3 (and a few that aren't), noting their status updated by C++23. Note that many of the adaptors here labelled C++20 or C++23 are in range-v3 also, we're just using the status "range-v3" to indicate that an adaptor is in range-v3 *only*:

View	Current Status	Proposed Priority
addressof	range-v3	Not proposed
adjacent	C++23	—
adjacent_transform	C++23	—
adjacent_filter	range-v3	Tier 2
adjacent_remove_if	range-v3	Tier 2
all	C++20	—
any_view<T>	range-v3	Not proposed
as_const	C++23	—
as_input	(not in range-v3)	Tier 1
as_rvalue	C++23	—
c_str	range-v3	Tier 1
cache1	range-v3	Tier 1. Possibly renamed as <code>cache_last</code> or <code>cache_latest</code>
cartesian_product	C++23	—
chunk	C++23	—
chunk_by	C++23	—
chunk_on	(not in range-v3)	Tier 1
common	C++20	—
concat	range-v3	Tier 1 [P2542R2]
counted	C++20	—
cycle	range-v3	Tier 1
delimit	range-v3	Tier 1
drop	C++20	—
drop_last	range-v3	Tier 1
drop_last_while	(not in range-v3)	Tier 1
drop_exactly	range-v3	Tier 1
drop_while	C++20	—
empty	C++20	—
enumerate	C++23	—
filter	C++20	—

View	Current Status	Proposed Priority
for_each	range-v3	Tier 1. Most languages call this flat_map, but we probably need to call it transform_join.
generate	range-v3	Tier 1
generate_n	range-v3	Tier 1
getlines	range-v3	Tier 1
group_by	range-v3	Not proposed. Subsumed by chunk_by.
head	(not in range-v3)	Tier 2
indirect	range-v3	Not proposed
intersperse	range-v3	Tier 2
ints	range-v3	Unnecessary unless people really hate iota.
iota	C++20	—
istream	C++20	See below for potential improvement.
iterate	(not in range-v3)	Tier 2
join	C++20	—
join_with	C++23	—
keys	C++20	—
linear_distribute	range-v3	Tier 3
maybe	proposed in [P1255R9]	???
partial_sum	range-v3	Tier 1, but not taking a callable (solely as a specialized form of scan)
remove	range-v3	Tier 1
remove_if	range-v3	Tier 1
repeat	C++23	—
repeat_n	C++23 (under the name repeat)	—
replace	range-v3	Tier 1
replace_if	range-v3	Tier 1
reverse	C++20	—

View	Current Status	Proposed Priority
sample	range-v3	Tier 3
scan	(not in range-v3)	Tier 1, as a rename of what is <code>partial_sum</code> in range-v3
set_difference	range-v3	Tier 3
set_intersection	range-v3	Tier 3
set_union	range-v3	Tier 3
set_symmetric_difference	range-v3	Tier 3
single	C++20	—
slice	range-v3	Tier 1
sliding	C++23 (as <code>slide</code>)	—
split	C++20 (improved)	—
split_when	range-v3	Tier 2
stride	C++23	—
tail	range-v3	Tier 2
take	C++20	—
take_exactly	range-v3	Tier 1
take_last	range-v3	Tier 1
take_last_while	(not in range-v3)	Tier 1
take_while	C++20	—
tokenize	range-v3	Not proposed
transform_filter	(not in range-v3)	Tier 1, related to <code>views::maybe</code> [P1255R9]
trim	range-v3	Tier 2
unbounded	range-v3	Not proposed
unique	range-v3	Tier 2
values	C++20	—
upto	not in range-v3	Tier 1 [P1894R0]
zip	C++23	—
zip_with	C++23	—

§ 2.1 `cache_last`

One of the adaptors that we considered for C++23 but ended up not pursuing was what range-v3 calls `cache1` and what we'd instead like to call something like `cache_last`. This is an adaptor which, as the name suggests, caches the last element. The reason for this is efficiency - specifically avoiding extra work that has to be done by iterator dereferencing.

The canonical example of this is `transform(f) | filter(g)`, where if you then iterate over the subsequent range, `f` will be invoked twice for every element that satisfies `g`:

```
int main()
{
    std::vector<int> v = {1, 2, 3, 4, 5};

    auto even_squares = v
        | std::views::transform([](int i){
            std::print("transform: {}\n", i);
            return i * i;
        })
        | std::views::filter([](int i){
            std::print("filter: {}\n", i);
            return i % 2 == 0;
        });

    for (int i : even_squares) {
        std::print("Got: {}\n", i);
    }
}
```

prints the following (note that there are 7 invocations of `transform`):

```
transform: 1
filter: 1
transform: 2
filter: 4
transform: 2
Got: 4
transform: 3
filter: 9
transform: 4
filter: 16
transform: 4
Got: 16
transform: 5
filter: 25
```

The solution here is to add a layer of caching:

```
auto even_squares = v
    | views::transform(square)
    | views::cache_last
    | views::filter(is_even);
```

Which will ensure that `square` will only be called once per element.

The tricky part here is: how do you implement `cache_last`? Specifically: in what member function do you perform the caching?

The range-v3 implementation looks roughly like this:

```

template <view V>
struct cache_last_view {
    V base_;
    bool dirty_ = true;
    non_propagating_cache<range_value_t<V>> cache_;

    struct iterator {
        cache_last_view* parent_;
        iterator_t<V> cur_;

        auto operator*() const -> range_value_t<V>&& {
            if (parent_ -> dirty_) {
                parent_ -> cache_.emplace(iter_move(cur_));
                parent_ -> dirty_ = false;
            }
            return std::move(*parent_ -> cache_);
        }

        auto operator++() -> iterator& {
            ++cur_;
            parent_ -> dirty_ = true;
        }
    };
};

```

But there's a problem here: 16.4.6.10 [\[res.on.data.races\]](#) says that `const` member functions are not allowed to introduce data races. While everything here is `const`-correct (there isn't even a `mutable`), iterator dereference here *does* introduce a data race: two threads were both dereferencing an iterator into a dirty `cache_last_view`.

There are four potential solutions to this problem, presented in our order of preference:

1. We could carve out an exception to [\[res.on.data.races\]](#) for all input iterators. Even some standard library implementations of input iterators (like `std::istreambuf_iterator<char>`) already don't satisfy this, and using input iterators in multi-threaded contexts is already kind of interesting. This makes the above implementation valid.
2. We could require synchronization on `operator*() const`. This probably isn't terrible expensive in this context, but adding synchronization to an adaptor whose primary purpose is to improve performance seems a bit heavy-handed, especially since that synchronization will almost never be actually necessary.
3. We could move the updating of the cached value from `operator*() const` to `operator++()`, which is already a mutable member function. This has the downside of requiring calculating more elements than necessary - since `r | cache_last | stride(2)` will still have to cache every element, even if only every other one is necessary.
4. We could allow input iterators to have *mutable* `operator*()`, since some of them clearly need it. A mutable `operator*()` makes the concepts even more awkward, and adds more work for every range adaptor. It theoretically is sensible, but seems extremely impractical.

The other issue is what the reference type of the range should be. range-v3 uses `range_value_t<V>&&`, but this somewhat defeats the purpose of caching if you can so easily invalidate it. `range_value_t<V>&` is probably a better choice.

§ 2.2 `istream<T>`

`views::istream<T>` was one of the original C++20 range factories, modified slightly since then to be a bit more user-friendly. But there's an interesting issue with it as pointed out in [P2406R5] and even before that in [range-v3#57]: `views::istream<T>(stream) | views::take(N)` will extract `N+1` elements from `stream`. Barry did a CppNow talk on this example ([video](#)).

There are, potentially, two approaches to implementing `views::istream<T>`:

Specified (C++20)	Alternative (as presented at CppNow)
<pre> template <class Val> class istream_view { istream* stream; Val value; struct iterator { istream_view* parent; auto operator++() -> iterator& { parent->extract(); return *this; } auto operator*() const -> Val& { return parent->value; } auto operator==(default_sentinel_t) const -> bool { return not *parent->stream; } }; auto extract() -> void { *stream >> value; } public: auto begin() -> iterator { extract(); return iterator{this}; } auto end() -> default_sentinel_t { return default_sentinel; } }; </pre>	<pre> template <class Val> class istream_view { istream* stream; Val value; struct iterator { istream_view* parent; mutable bool dirty = true; auto prime() const -> void { if (dirty) { *parent->stream >> parent->value; dirty = false; } } auto operator++() -> iterator& { prime(); dirty = true; return *this; } auto operator*() const -> Val& { prime(); return parent->value; } auto operator==(default_sentinel_t) const -> bool { prime(); return not *parent->stream; } }; public: auto begin() -> iterator { return iterator{this}; } </pre>

Specified (C++20)	Alternative (as presented at CppNow)
	<pre> } auto end() -> default_sentinel_t { return default_sentinel; } }; </pre>

This alternative implementation ensures that consuming `views::istream<T>(stream) | views::take(N)` extracts exactly `N` elements from `stream`, including for `N == 0`. It does, however, require doing work in two different `const` member functions: both `operator*()` and `operator==()`. Neither of these violate the semantic guarantees of those functions - repeated invocations of either will give you the same result every time, until you increment again. But they do violate 16.4.6.10 [\[res.on.data.races\]](#).

We have the same potential four options here as we described with [cache_last](#), but we could also just keep the existing implementation of `views::istream<T>`. Changing this range does have observable effects, but we think we should seriously consider doing so. LEWG seemed very willing to change `counted_iterator<I>` and `views::take` in order to address this issue before, so we think serious consideration should be given to changing `views::istream<T>`.

Additionally, this would set a precedent for how to write these kinds of input ranges. So it's important to get right.

Separately, there is also `views::getlines`. In the same way that `views::istream<T>(is)` is a factory that produces elements of type `T` on demand by way of `is >> obj`, `views::getlines` is a factory that produces elements of type `std::string` on demand by way of `std::getline(is, obj)`. Note that both could nearly be implemented in terms of `views::generate`:

<code>views::istream<T></code>	<code>views::getlines</code>
<pre> template <class T> inline constexpr auto istream = [] (std::istream& is){ return views::generate([&is, obj=T()]() mutable -> T& { is >> obj; return obj; }); }; </pre>	<pre> inline constexpr auto getlines = [] (std::istream& is, char delim = '\n'){ return views::generate([&is, delim, obj=std::string()]() mutable -> std::string& { std::getline(is, obj); return obj; }); }; </pre>

Almost because neither of these terminates, and we eventually do need some kind of termination condition. Which might call for some kind of `views::generate_until`.

§ 2.3 scan

If you want to take a range of elements and get a new range that is applying `f` to every element, that's `transform(f)`. But there are many cases where you need a transform to that is *stateful*. That is, rather than have the input to `f` be the current element (and require that `f` be `regular_invocable`), have the input to `f` be both the current element *and* the current state.

For instance, given the range `[1, 2, 3, 4, 5]`, if you want to produce the range `[1, 3, 6, 10, 15]` - you can't get there with `transform`. Instead, you need to use `scan` using `+` as the binary operator. The special case of `scan` over `+` is `partial_sum`.

One consideration here is how to process the first element. You might want `[1, 3, 6, 10, 15]` and you might want `[0, 1, 3, 6, 10, 15]` (with one extra element), the latter could be called a *prescan*.

§ 2.4 generate

C++23 has `std::generator<T>`. There are two very closely related range factories in `range-v3`, which are basically:

```
template <class F>
    requires std::invocable<F&>
auto generate(F f) -> std::generator<std::invoke_result_t<F&>> {
    while (true) {
        co_yield f();
    }
}

template <class F>
    requires std::invocable<F&>
auto generate_n(F f, int n) -> std::generator<std::invoke_result_t<F&>> {
    for (int i = 0; i != n; ++i) {
        co_yield f();
    }
}
```

Note that the constraint here is `invocable`, not `regular_invocable`. The latter wouldn't be very interesting - that's `views::repeat(f())`. These factories are somewhat related to `scan` (in the sense that we have a mutable function that we're repeatedly invoking) and also somewhat related to `cache_latest` (in the sense that the `range-v3` implementation of both also violate `[res.on.data.races]`).

Since with `views::repeat`, we just used the same name for the infinite and finite versions, we should probably end up with just the one name for `views::generate`.

A similar factory in this vein is one that Haskell calls `iterate`:

```
template <class F, class T>
auto iterate(F f, T x) -> std::generator<T> {
    while (true) {
        co_yield x;
        x = f(x);
    }
}
```

Whereas `generate(f)` is the sequence `[f(), f(), f(), f(), ...]`, `iterate(f, x)` is the sequence `[x, f(x), f(f(x)), f(f(f(x))), ...]`

Yet another factory, following the theme, is one that Dlang calls recurrence ([implementation](#)). Although maybe this one is too cute:

```
auto main() -> int {
    // fibonacci: [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
    print("fibonacci: {}\n",
        recurrence([](auto a, int n){ return a[n-1] + a[n-2]; }, 1, 1)
        | views::take(10)
    );

    // factorial: [1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880]
    print("factorial: {}\n",
        recurrence([](auto a, int n){ return a[n-1] * n; }, 1)
        | views::take(10)
    );
}
```

§ 2.5 as_input

We added two fairly simply adaptors in C++23: `views::as_const` and `views::as_rvalue`, both of which are specialized versions of `views::transform`. Well, `views::as_const` is conceptually simple anyway - even as it is remarkably complex.

There's a third adaptor in this family that we should consider adding: `views::as_input(r)`. This is an adaptor that all it does is reduce `r`'s category to input and force it to be non-common. Otherwise: same value type, same reference type, same sized-ness, same borrowed-ness, same const-iterability.

Why would anybody want such a thing? Performance.

Range adaptors typically provide the maximum possible iterator category - in order to maximize functionality. But sometimes it takes work to do so. A few examples:

- `views::join(r)` is common when `r` is, which means it provides two iterators. The iterator comparison for `join` does [two iterator comparisons](#), for both the outer and the inner iterator, which is definitely necessary when comparing two iterators. But if all you want to do is compare it `==` end, you could've gotten away with [one iterator comparison](#). As such, iterating over a common `join_view` is more expensive than an uncommon one.
- `views::chunk(r, n)` has a different algorithm for input vs forward. For `forward+`, you get a range of `views::take(n)` - if you iterate through every element, then advancing from one chunk to the next chunk requires iterating through all the elements of that chunk again. For input, you can only advance element at a time.

The added cost that `views::chunk` adds when consuming all elements for `forward+` can be necessary if you need the forward iterator guarantees. But if you don't need it, like if you're just going to consume all the elements in order one time. Or, worse, the next adaptor in the chain reduces you down to input anyway, this is unnecessary.

In this way, `r | views::chunk(n) | views::join` can be particularly bad, since you're paying additional cost for chunk that you can't use anyway, since `views::join` here would always be an input range. `r | views::as_input | views::chunk(n) | views::join` would alleviate this problem. It would be a particularly nice way to alleviate this problem if users didn't have to write the `views::as_input` part!

This situation was originally noted in [\[range-v3#704\]](#).

§ 2.6 Simple Adaptor Compositions

Many adaptors have to have their own dedicated implementation. Some are merely more convenient spellings of existing ones (like `keys` for `elements<0>` and `pairwise` for `adjacent<2>`). Still others could be just compositions of existing range adaptors.

One such is what most of the rest of the world calls `flat_map`: this is a combination of `map` and then `flatten`. In C++ terms, we could very simply provide such an adaptor:

```
inline constexpr auto transform_join = []<class F>(F&& f){
    return transform((F&&)f) | join;
};
```

Well, the actual implementation is slightly more involved in order to be able to also support `views::transform_join(r, f)` in addition to `r | views::transform_join(f)`, but not dramatically so. Importantly, there really isn't much benefit to providing a bespoke `transform_join` as opposed to simply implementing it in terms of these two existing adaptors. But this is such a common piece of functionality that it probably merits direct addition into the standard library.

In slide-ware, it probably doesn't make that much of a difference. But in real code that uses namespaces, it really does:

```
r | transform(f) | join
r | transform_join(f)

r | std::views::transform(f) | std::views::join
r | std::views::transform_join(f)
```

A few other common patterns worth considering:

- `views::replace(old_val, new_val)` and `views::replace_if(pred, new_val)` are kinds of `views::transform`
- `views::remove(val)` and `views::remove_if(pred)` are kinds of `views::filter`, the latter being just `filter(not_fn(pred))`
- `views::upto(n)` is just `views::iota(dec<type>(n){}, n)`, which is useful not just because it's terser and a better name, but also because a fairly typical use is `views::iota(0, r.size())` - or at least it would be, but that doesn't compile when `r.size()` is unsigned.
- For the [algorithms](#) discussed later, `ranges::sum` and `ranges::product` are just special cases of `ranges::reduce`.

But it is not always the case that just writing one algorithm in terms of others is optimal. It is tempting to define `views::tail` as simply `views::drop(1)`, but a dedicated `tail` could be more efficient (it does not need to store the count or cache `begin()`). It's unfortunate that the relative difference in specification is so high though.

§ 2.7 Extending conditionally borrowed

In [P2017R1], we made some range adaptors conditionally borrowed. But we didn't touch adaptors that had callables - like `views::transform`. It turns out to be very useful to have a borrowable version of `views::transform`. Indeed, [P2728R6] even adds a dedicated new range adaptor (`views::project`) which is simply a version of `views::transform` that can be borrowed (because its callable must be a constant).

But rather than add a dedicated view for this specific case, which requires a new name but really only helps `views::transform`, we can generalize `views::transform` to address the use-case in a way that would also help all the other range adaptors that take callables. At the very least, in `views::transform(r, f)` if `r` is borrowed and `f` is empty, an implementation can simply put `f` in the `transform_view<R, F>::iterator` directly (rather than a `transform_view<R, F>*`) which would allow it to be borrowed. The same could be said for other range adaptors that take callables as well, which seems like a more useful approach as well as not requiring new names for every adaptor.

The main question then is what the criteria should be for when `transform_view<R, F>` should be a borrowed range (when `R` is):

- `is_empty_v<F>` (range-v3 already does this - not for conditionally borrowed, but just to decide whether to store the callable by value in the iterator)
- `sizeof(F) <= sizeof(void*)` and `is_trivially_copyable_v<F>` (this means that when transforming with a function pointer, the function pointer itself can live in the iterator - which takes the same amount of space as the parent pointer, except with one less indirection)
- something else?

This question is a little simpler for `views::transform` (which only needs to potentially store `f` in the adapted iterator) than it is for `views::filter` (which would need not only the predicate but also the underlying sentinel, so this may not be worthwhile). This would need to be carefully considered.

§ 3 View Adjuncts

In the C++23 plan, we listed several facilities that would greatly improve the usability of views: the ability for users to define first class pipe support, the ability to collect into a container (`ranges::to`), and formatting.

There are some other operations that we've seen come up regularly - operations that are not themselves views or algorithms, but would improve the quality of life around using the standard library (and other) range adaptors.

§ 3.1 More Function Objects

The standard library has a lot of function objects, but there are still plenty of common ones that are missing.

Some unary operators have no associated function object:

- indirection: `*_1`
- addressof: `&_1` (except if we add a function object for this, it should do `std::addressof`)
- prefix and postfix increment: `++_1` or `_1++`
- prefix and postfix decrement: `--_1` or `_1--`

range-v3 has `views::indirect`, for instance, which is basically an over-constrained `views::transform(*_1)`.

Some binary operators have no associated function object:

- the shifts: `_1 << _2` and `_1 >> _2`
- all the compound assignments: `_1 += _2`, etc.

The various language cases also have no associated function object. The most common of these is `static_cast<T>(_1)`.

It is also worth considering whether we should actually add function objects for these, like `std::indirect` (or `std::ranges::indirect`?) or whether we should try to bring back one of the earlier proposals that added nicer syntax for passing operators as function objects:

Paper	Syntax
[P0119R2]	<code>views::transform((*))</code>
[P0834R0]	<code>views::transform([] *)</code>
[P2672R0] (placeholder lambda)	<code>views::transform([] *\$1)</code> <code>views::transform([] \$(\$1))</code>
backticks	<code>views::transform(`*`)</code>

§ 3.2 More Function Adaptors

The standard library doesn't have very many function adaptors. There are two particularly notable ones that seem to come up frequently.

- function composition: an adaptor `compose` such that `compose(f, g)(x...) == f(g(x...))`
- function projection: an adaptor `proj` such that `proj(p, f)(x...) == f(p(x)...)`

If we had a `proj` adaptor, people wouldn't need to ask for range adaptors to support projections - they could just provide one.

The difficulty with these is that both are syntactically heavy in C++, because our lambdas are verbose and we have difficulties passing functions around (see the two papers noted in the previous section).

The other problem is that these adaptors don't really have obvious ordering. Should `compose(f, g)(x)` mean `f(g(x))` or `g(f(x))`? There's good arguments for either. The same is true for `proj` (which is sometimes also called `on`).

§ 4 Algorithms

We improved the Ranges story on algorithms quite a bit in C++23 - both in terms of new and existing algorithms. But there's a few more pretty interesting ones left on the table.

§ 4.1 reduce

We talked about [reduce](#) in [P2214R2]. `ranges::reduce` is a version of `ranges::fold_left` ([P2322R6]) that is parallelizable. It requires the binary operation to be associative (to allow chunks of the range to be reduced in parallel) and commutative (to allow those chunks to be arbitrarily combined). So we will need to figure out what constraints to add on this algorithm (see [P1813R0]) as well as how we determine what the return type is (see [this section](#) discussing the same problem for `ranges::fold_left`).

One thing is clear: `ranges::reduce` should *not* take a default binary operation *nor* a default initial parameter. The user needs to supply both.

However, for convenience, we do propose providing `ranges::sum(r)` as `ranges::reduce(r, plus{}, range_value_t<R>())` and `ranges::product(r)` as `ranges::reduce(r, multiplies{}, range_value_t<R>(1))`.

Note that naming is a problem here: some languages (Rust, Scala, Kotlin) have an algorithm that takes an initial value named `fold` and an algorithm that takes no initial value and returns an optional `reduce`. In C++23, we called these `fold_left` and `fold_left_first` since we've already had `std::reduce` since C++17.

But since our `reduce` differs from our `fold` not based on initial element but rather on operation requirements, it also leaves open the question for whether there should be a `reduce_first`. A good example there might be using `std::max` as the reduction operator - which is both associative and commutative, but for some types may not have an obvious choice for the minimum.

§ 4.2 distance and advance

We have `ranges::size(E)`, which gives you the size of a range in constant time. For non-sized ranges, if you want to know the size you have to use `ranges::distance(E)`. For non-sized ranges though, `ranges::distance` has to iterate over the entire range, element by element, counting the number of iterator increments until the sentinel is reached.

For many ranges, that's really the best you can do anyway. But for some, you could do better. Consider `views::join`. You could, potentially, do *much* better on distance in some cases: if I'm joining a range of sized ranges (like `vector<vector<T>>`, although the outer one need not be sized, so even `forward_list<vector<T>>`), you could compute the size of the overall range by summing the `size()` of each element. That's still not `O(1)`, so `ranges::size` cannot do this, but it would be substantially more efficient than the naive `ranges::distance` implementation.

A similar argument holds for `ranges::advance` for non-random-access iterators. Implementations already do provide special-case overloads for `std::advance` in some cases, though they cannot do so for `ranges::advance`. For instance, `libstdc++` provides a [custom implementation](#) for `std::istreambuf_iterator<Char>`. You cannot provide it `+ n`, because that cannot necessarily be constant time, but `advance` doesn't have to be constant - it just has to get there (reduced for brevity):

```
template<typename _CharT, typename _Distance>
advance(istreambuf_iterator<_CharT>& __i, _Distance __n)
{
    if (__n == 0)
        return;

    using traits_type = /* ... */;
    const traits_type::int_type __eof = traits_type::eof();

    streambuf_type* __sb = __i._M_sbuf;
    while (__n > 0) {
        streamsize __size = __sb->egptr() - __sb->gptr();
        if (__size > __n) {
            __sb->_M_in_cur += __n;
            break;
        }

        __sb->_M_in_cur += __size;
        __n -= __size;
        if (traits_type::eq_int_type(__sb->underflow(), __eof)) {
            break;
        }
    }

    __i._M_c = __eof;
}
```

The advance here is that if we want to `advance(it, 10)`, we can simply right away check if there are at least 10 characters in the get area. If there are, we just advance by 10 and we're done. If not, we have to go pull more characters. Either way, we end up significantly reducing the number of times that we have to go back to the stream - we're not pulling one character at a time, we're potentially consuming the entire get buffer at a time, for a significant reduction in the number of branches.

This is more efficient for the same reason that the hypothetical implementation of `ranges::distance` for a `join_view` could be more efficient.

Currently, none of the non-constant-time algorithms (like `distance`, `advance`, and `next`) are customizable - but there could be clear benefits to making them so. Unfortunately, there are very clear

costs to making them so: even more work that every range and iterator adaptor has to do.

§ 5 Output Iterators

There are two kinds of output iterators: those that are also input iterators (like `int*`) and those that are not. This section is dedicated to output-only iterators. The one of these that people are probably most familiar with is `std::back_insert_iterator<C>`.

Output-only iterators are important, yet severely underpowered. The problem with them ultimately is they are shoe-horned into the same *syntax* as input iterators, despite not really have anything to do with iterators.

If we take an algorithm like `std::copy`, it's implemented something like this:

```
template <typename InputIt, typename OutputIt>
void copy(InputIt first, InputIt last, OutputIt out) {
    for (; first != last; ++first) {
        *out++ = *first;
    }
}
```

In order to provide `std::back_insert_iterator<C>`, it has to meet that syntax. So we end up with something like:

```
template <typename C>
class back_inserter {
    C* cont_;

public:
    explicit back_inserter(C& c) : cont_(&c) { }

    // these do nothing
    auto operator*() -> back_inserter& { return *this; }
    auto operator++() -> back_inserter& { return *this; }
    auto operator++(int) -> back_inserter { return *this; }

    // this one does something
    auto operator=(typename C::value_type const& val) -> back_inserter& {
        cont_>push_back(val);
        return *this;
    }

    // same
    auto operator=(typename C::value_type&& val) -> back_inserter& {
    };
};
```

There are two problems with this approach. First, it's a really awkward API to go about implementing an output iterator. You have to write three no-op functions and one useful function, whose spelling doesn't really convey any meaning. An output-only iterator *is* a function call, yet it cannot be implemented as such, which is an annoying loss in convenience since you cannot simply use a lambda as an output

iterator. Sure, it's not a huge task to implement a `function_output_iterator<F>` - you can find such a thing [in Boost](#) too - but there really shouldn't be a need for this.

But more importantly, it's very inefficient. An output-only iterator gets one element at a time, even when the algorithm knows it's producing more. A common use of `back_inserter` is doing something like this:

```
std::vector<T> vec;
std::ranges::copy(r, std::back_inserter(vec));
```

That will compile into N calls to `vec.push_back`. Maybe `r` is an unsized input range and that's the best you can do anyway. But if `r` is sized, that's pretty wasteful - `vector` has a range insertion API which does the right thing, it can be much more efficient to simply call:

```
std::vector<T> vec;
vec.append_range(r);
```

Indeed, 2.7x faster in [this simple benchmark](#).

This is a known problem, to the point where libraries try to detect and work around this pessimization. The `{fmt}` formatting library, now `<format>` since C++20, is entirely output-iterator based. But, because of type erasure, the typical output iterator that you will interact with is an output-only iterator, not an input iterator. So what happens when you try to write a `std::string_view` through that output iterator (a not-especially-uncommon operation when it comes to formatting)?

`{fmt}` has an internal helper named `copy_str`, whose [default implementation](#) is pretty familiar:

```
template <typename Char, typename InputIt, typename OutputIt>
FMT_CONSTEXPR auto copy_str(InputIt begin, InputIt end, OutputIt out)
    -> OutputIt {
    while (begin != end) *out++ = static_cast<Char>(*begin++);
    return out;
}
```

But there's this other [important overload too](#):

```
template <typename Char, typename InputIt>
auto copy_str(InputIt begin, InputIt end, appender out) -> appender {
    get_container(out).append(begin, end);
    return out;
}
```

For most of the operations in `{fmt}`, the implementation-defined type-erased iterator is `appender`, so this would be the overload used. And `appender` is a `back_inserter` into a `buffer<char>`, which is a growable buffer (not unlike `vector<char>`) which has a [dedicated append](#) for this case:

```
template <typename T>
template <typename U>
void buffer<T>::append(const U* begin, const U* end) {
    while (begin != end) {
        auto count = to_unsigned(end - begin);
        try_reserve(size_ + count);
```

```

auto free_cap = capacity_ - size_;
if (free_cap < count) count = free_cap;
std::uninitialized_copy_n(begin, count, make_checked(ptr_ + size_, count));
size_ += count;
begin += count;
}
}

```

So here, we know that `std::copy` and `std::ranges::copy` would be inefficient, so the library provides (and internally uses) a way to special case that algorithm for its particular output iterator.

This kind of thing really shouldn't be QoI. Output-only iterators that can support efficient range-based operations should be able to do so.

§ 5.1 Potential Design

Barry laid out an approach in a blog post [[improve.output](#)] based on the model the D library uses, using two customization point objects: one for single elements and one for a range of elements:

`ranges::put(out, e)` could be the first valid expression of:

1. `out.put(e)`
2. `*out++ = e;`
3. `out(e);`

`ranges::put_range(out, r)` could be the first valid expression of:

1. `out.put_range(r)`
2. `ranges::for_each(r, bind_front(ranges::put, out))`

This isn't quite what D does, but it's more suited for C++, and would allow output-only iterators to be as efficient (and easy to implement) as they should be.

If we had the above, the implementation of `back_insert_iterator` would become:

```

template <typename C>
class back_inserter {
    C* cont_;

public:
    explicit back_inserter(C& c) : cont_(&c) { }

    auto put(typename C::value_type const& val) -> void {
        cont_>push_back(val);
    }
    auto put(typename C::value_type&& val) -> void {
        cont_>push_back(std::move(val));
    }
}

```

```

template <ranges::input_range R>
    requires std::convertible_to<ranges::range_reference_t<R>, typename
        C::value_type>
auto put_range(R&& r) -> void
{
    if constexpr (requires { cont_>append_range(r); }) {
        cont_>append_range(r);
    } else if constexpr (requires { cont_>insert(cont_>end(),
        ranges::begin(r), ranges::end(r)); }) {
        cont_>insert(cont_>end(), ranges::begin(r), ranges::end(r));
    } else {
        for (auto&& e : r) {
            cont_>push_back(FWD(e));
        }
    }
}
};

```

Sure, `put_range` is mildly complicated, but it's much more efficient than the original implementation, and we no longer have functions that do nothing.

Now, the issue here is that this is a fairly large redesign of the output iterator model with minimal implementation experience (unless you count D or the blog post). So this approach needs more time, but we do think it's worth doing.

§ 6 Plan Summary

As previously, we want to triage a lot of outstanding views, algorithms, and other utilities into three tiers based on our opinions of their importance. While ideally we could just add everything into C++26, we realize that this is not realistic with the amount of available LWG bandwidth, so our tier 1 here is trying to be as small as possible while still hitting as many major pain points as possible.

§ 6.1 Tier 1

- Range Adaptors:
 - `views::concat` ([P2542R2])
 - Take/Drop Family:
 - `views::drop_last` and `views::take_last`
 - `views::drop_last_while` and `views::take_last_while`
 - `views::drop_exactly` and `views::take_exactly`
 - `views::slice`
 - Simple Adaptor Compositions:
 - `views::transform_join`
 - `views::replace` and `views::replace_if`
 - `views::remove` and `views::remove_if`

- `views::upto`
- `views::as_input`
- `views::cache_last`
- `views::chunk_on`
- `views::cycle`
- `views::delimit` and `views::c_str`
- making more adaptors (e.g. `views::transform`) conditionally borrowed in more complex circumstances
- Generators:
 - `views::scan`
 - `views::generate` and `views::generate_n`
- Algorithms:
 - `ranges::reduce`
 - `ranges::sum`
 - `ranges::product`

§ 6.2 Tier 2

- Range Adaptors:
 - `views::adjacent_filter`
 - `views::adjacent_remove_if`
 - `views::head`
 - `views::intersperse`
 - `views::iterate`
 - `views::split_when`
 - `views::tail`
 - `views::trim`
 - `views::unique`

§ 6.3 Tier 3

- Range Adaptors:
 - `views::linear_distribute`
 - `views::sample`
- Set Adaptors

- `views::set_difference`
- `views::set_intersection`
- `views::set_symmetric_difference`
- `views::set_union`

§ 7 References

- [improve.output] Barry Revzin. 2022-02-06. Improving Output Iterators.
<https://brevzin.github.io/c++/2022/02/06/output-iterators/>
- [P0119R2] Andrew Sutton. 2016-05-28. Overload sets as function arguments.
<https://wg21.link/p0119r2>
- [P0834R0] Michael Dominiak. 2017-10-16. Lifting overload sets into objects.
<https://wg21.link/p0834r0>
- [P1206R7] Corentin Jabot, Eric Niebler, Casey Carter. 2022-01-21. Conversions from ranges to containers.
<https://wg21.link/p1206r7>
- [P1255R9] Steve Downey. 2022-08-16. A view of 0 or 1 elements: `views::maybe`.
<https://wg21.link/p1255r9>
- [P1813R0] Christopher Di Bella. 2019-08-02. A Concept Design for the Numeric Algorithms.
<https://wg21.link/p1813r0>
- [P1894R0] Andrew Tomazos. 2019-10-02. Proposal of `std::upto`, `std::indices` and `std::enumerate`.
<https://wg21.link/p1894r0>
- [P1899R3] Christopher Di Bella, Tim Song. 2022-07-11. `stride_view`.
<https://wg21.link/p1899r3>
- [P2017R1] Barry Revzin. 2020-02-19. Conditionally borrowed ranges.
<https://wg21.link/p2017r1>
- [P2164R9] Corentin Jabot. 2022-12-07. `views::enumerate`.
<https://wg21.link/p2164r9>
- [P2214R2] Barry Revzin, Conor Hoekstra, Tim Song. 2022-02-18. A Plan for C++23 Ranges.
<https://wg21.link/p2214r2>
- [P2278R4] Barry Revzin. 2022-06-17. `cbegin` should always return a constant iterator.
<https://wg21.link/p2278r4>
- [P2286R8] Barry Revzin. 2022-05-16. Formatting Ranges.
<https://wg21.link/p2286r8>

- [P2302R4] Christopher Di Bella. 2022-04-17. `std::ranges::contains`.
<https://wg21.link/p2302r4>
- [P2321R2] Tim Song. 2021-06-11. `zip`.
<https://wg21.link/p2321r2>
- [P2322R6] Barry Revzin. 2022-04-22. `ranges::fold`.
<https://wg21.link/p2322r6>
- [P2374R4] Sy Brand, Michał Dominiak. 2022-07-13. `views::cartesian_product`.
<https://wg21.link/p2374r4>
- [P2387R3] Barry Revzin. 2021-12-17. Pipe support for user-defined range adaptors.
<https://wg21.link/p2387r3>
- [P2406R5] Yehezkel Bernat, Yehuda Bernat. 2023-02-08. Add `lazy_counted_iterator`.
<https://wg21.link/p2406r5>
- [P2408R5] David Olsen. 2022-04-22. Ranges iterators as inputs to non-Ranges algorithms.
<https://wg21.link/p2408r5>
- [P2440R1] Tim Song. 2021-12-06. `ranges::iota`, `ranges::shift_left`, and `ranges::shift_right`.
<https://wg21.link/p2440r1>
- [P2441R2] Barry Revzin. 2022-01-28. `views::join_with`.
<https://wg21.link/p2441r2>
- [P2442R1] Tim Song. 2021-12-06. Windowing range adaptors: `views::chunk` and `views::slide`.
<https://wg21.link/p2442r1>
- [P2443R1] Tim Song. 2021-11-19. `views::chunk_by`.
<https://wg21.link/p2443r1>
- [P2446R2] Barry Revzin. 2022-02-15. `views::as_rvalue`.
<https://wg21.link/p2446r2>
- [P2474R2] Michał Dominiak. 2022-07-13. `views::repeat`.
<https://wg21.link/p2474r2>
- [P2542R2] Hui Xie, S. Levent Yilmaz. 2022-05-11. `views::concat`.
<https://wg21.link/p2542r2>
- [P2672R0] Barry Revzin. 2022-10-14. Exploring the Design Space for a Pipeline Operator.
<https://wg21.link/p2672r0>
- [P2728R6] Zach Laine. 2023-07-11. Unicode in the Library, Part 1: UTF Transcoding.
<https://isocpp.org/files/papers/P2728R6.html>

[range-v3#57] Eric Niebler. 2014. istream_range filtered with take(N) should stop reading at N.
<https://github.com/ericniebler/range-v3/issues/57>

[range-v3#704] Eric Niebler. 2017. Demand-driven view strength weakening.
<https://github.com/ericniebler/range-v3/issues/704>